

SDEV2171

Lesson 15: Sharing State Between Screens

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, you should be able to:

- Explain why state passed through route params drifts out of sync
- Show which screens stay mounted in a stack, and why that matters for state
- Sort a value as local, duplicated, or shared, and decide where it should live
- Move shared state into a context provider in the root layout, and read it with a custom hook
- Refactor in small steps that keep the app running after each one
- Describe how production apps separate server state from client state

WE ARE ESSENTIAL TO ALBERTA | NAIT

Today

Part	Block	Minutes
1	Run the backend and the starter app	8
1	Reproduce the problem	7
1	Why the screens drift apart	13
1	Where state should live	10
1	Checkpoint	2
	Break	10
2	Context, and the order we’ll build it in	5
2	Moving the plan into context	29
2	How production apps do this	6
2	Challenges and checkpoint	10

WE ARE ESSENTIAL TO ALBERTA | NAIT

Run the Backend and the Starter App

The backend is the one from lesson 13. In its own terminal, run `node server.js` from the `backend` folder, and leave it running. The Study Steps screen requests `/api/study-steps?delay=500`, so its list appears after half a second.

In a second terminal, from the `starter-app` folder:

```
npm install
npm start
```

Press `w` to open it in the browser, or open your development build. The app's config and dependencies are the same as lesson 13's, so the development build you made for lesson 13 still works.

`src/lib/apiBaseUrl.js` is unchanged: `10.0.2.2` on the Android emulator, `localhost` everywhere else, and your computer's LAN IP on a phone.

WE ARE ESSENTIAL TO ALBERTA | NAIT

What the Starter App Does

Screen	What it does
Home	opens Study Steps
Study Steps	fetches the steps with `useFetch` from lesson 13. The Add and Remove button on each row builds a plan, and the count at the top shows its size. Open plan sends the plan to the Plan screen
Plan	lists the steps it was sent. Remove takes one out

The plan is a `useState` array in `details.js`. The Plan screen receives it through route params.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Two Screens Disagree

Do these in order, and read the count after each one:

Do this	Study Steps shows	Plan shows
Add three steps	3 in your plan	
Press Open plan		3 in your plan
Remove one step		2 in your plan
Go back	3 in your plan	
Press Open plan again		3 in your plan

The removal is lost. Study Steps never found out about it, and the second Open plan sent its own list again.

Go back with the app’s back arrow or Go back button, not the browser’s Back button.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Route Params Are Strings

Study Steps sends the plan like this:

```
router.push({ pathname: '/plan', params: { steps: JSON.stringify(selectedSteps) } });
```

Expo Router puts params in the URL, so a param can only be a string. To send an array, the screen converts it to JSON. In the browser, open the plan and read the address bar. Decoded, it says:

`/plan?steps=[{"id":"1","label":"Charge laptop before class"},{"id":"2",...}]`

The Plan screen receives a copy of the list as it was at the moment you pressed Open plan. Nothing connects that copy to the array in Study Steps.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Plan Screen Keeps Its Own Copy

```
const { steps } = useLocalSearchParams();
// The useState initializer runs once, when this screen mounts.
const [planSteps, setPlanSteps] = useState(() => JSON.parse(steps ?? '[]'));
```

The function passed to `useState` runs once, when the screen mounts. After that, `planSteps` belongs to the Plan screen, and `removeStep` changes only the Plan screen's copy.

The app now has two arrays that each claim to be the plan. That is **duplicated state**: two owners of one piece of data, and nothing keeping them in step.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Stack Keeps Study Steps Mounted

Pushing a screen puts it on top of the stack, and the screen underneath stays mounted with its state. Going back unmounts the top screen.

Check it. Add this to `details.js` temporarily, and the same with `'Plan'` to `plan.js`:

```
useEffect(() => {
  console.log('Details mounted');
  return () => console.log('Details unmounted');
}, []);
```

Open Study Steps, open the plan, then go back. The console shows `Details mounted`, `Plan mounted`, `Plan unmounted`, and never `Details unmounted`.

Study Steps keeps its list the whole time. Plan starts fresh each time, from its params.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why the Screens Drift Apart

Three facts combine:

1. Route params are a string snapshot, taken when you navigate
2. The Plan screen copies that snapshot into its own state once, when it mounts
3. Study Steps stays mounted underneath, holding its own list

Each screen changes only its own copy, and neither can see the other’s changes.

Sending the plan back to Study Steps with more params doesn’t fix this. Every screen that shows the plan would need a copy sent to it, and each copy is another snapshot that can go stale. A third screen that shows the plan makes it worse.

WE ARE ESSENTIAL TO ALBERTA | NAIT

What Route Params Are For

Params are good for telling a screen **which** thing to show. They’re poor at carrying the thing itself.

A good param	Why it works
the `3` in `/step/3`	an id. The screen looks up step 3 itself, so it always shows current data
`q=che` in `/details?q=che`	a search the user could bookmark or share
a title for the next screen’s header	read-only, and harmless if it’s out of date

On the web and in deep links, the URL is all a screen gets. A bookmarked `/plan?steps=[...]` shows a plan that may not exist anymore. A bookmarked `/step/3` still works.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Local, Duplicated, and Shared State

Kind	Meaning	In the starter app
Local	one component owns it, and only that component uses it	`status` and `errorMessage` inside `useFetch`
Duplicated	two components each keep a copy of the same data	`selectedSteps` in Study Steps, and `planSteps` in Plan
Shared	one owner, and every component that needs it reads it from that owner	the goal for the plan

Local state isn’t a problem, and most state should stay local. Duplicated state is the problem.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Where Each Value Belongs

For each value, ask: if it changes on this screen, does another screen need to see the change?

Value	Does another screen need it?	Where it lives
`status` and `errorMessage` for the request	No	local, in `useFetch`
the fetched study steps	No. Only Study Steps lists them	local, in `useFetch`
the plan	Yes: Study Steps, Plan, and soon Home	shared
whether one row is selected	It’s computed from the plan	nowhere. It isn’t state

The last row is a **derived value**. Compute it during render from state you already have, instead of storing it. Storing it would make a third copy.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lifting State Up

React’s standard fix for two components that need the same state is to **lift the state up**: move it into their closest common parent, and pass it down as props. The parent owns the state. The children read it from props, and change it by calling functions the parent passes down.

In this app, the closest common parent of Study Steps and Plan is the root layout, `src/app/_layout.tsx`. Every screen renders inside it.

So the plan’s state belongs in the layout, or in a component the layout renders around the screens.

WE ARE ESSENTIAL TO ALBERTA | NAIT

A Layout Can’t Pass Props to Its Screens

```
<Stack>
  <Stack.Screen name="index" options={{ title: 'Home' }} />
  <Stack.Screen name="details" options={{ title: 'Study Steps' }} />
  <Stack.Screen name="plan" options={{ title: 'Plan' }} />
</Stack>
```

Nowhere in the app do you write `<DetailsScreen steps={...} />`. Expo Router renders each screen from its file, so the layout has nowhere to pass props.

The layout can own the state, but it needs another way to hand it down. That is what **Context** does: a component above the screens provides a value once, and any component below it reads the value directly, without props.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Part 1 Checkpoint

- Why doesn’t removing a step on the Plan screen change the count on Study Steps?
- Which screen stays mounted when you open the plan, and how did you check?
- Name one value in this app that should stay local, and one that should be shared
- Give one thing route params are good for, and one thing they’re bad for

Before Part 2, remove the temporary mount logs from `details.js` and `plan.js`. They aren’t committed.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Context Shares a Value Without Props

```
const StudyPlanContext = createContext(null);

// A component above the screens provides the value:
<StudyPlanContext value={value}>{children}</StudyPlanContext>

// Any component below it reads the value:
const value = useContext(StudyPlanContext);
```

Piece	Its job
`createContext(defaultValue)`	creates the context. The default is used only with no provider above
the provider	supplies the current value to every component inside it
`useContext(SomeContext)`	reads the value from the nearest provider above

React 19 renders the context itself as the provider. Older code writes `<StudyPlanContext.Provider>`, which React plans to deprecate.

WE ARE ESSENTIAL TO ALBERTA | NAIT

How to Build and Commit a Change

Build every change in this order:

1. Write the shape first, with hard-coded values
2. Connect everything that uses it
3. Check that the app runs and shows the hard-coded values
4. Replace the hard-coded values with the real logic
5. Test the whole flow
6. Commit

In today’s refactor, the shape is the context’s value: what it holds, and which functions it offers. Once both screens read that shape, replacing the hard-coded values with real state changes one file and no screens.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Moving the Plan Into Context

The refactor is four small changes, and the app runs after each one:

Change	What you check after it
Add the context file with a hard-coded value, and wrap the layout in its provider	the app works exactly as before
Read the plan from context on both screens	both screens show 0, and Add does nothing yet
Keep the plan’s state in the provider	both screens agree
Show the plan count on Home	a third screen agrees too

The second row looks like a step backwards, and that’s on purpose. The screens are connected to the context before the context holds anything real, so if a screen shows the wrong thing later, the bug is in the provider.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Context’s Contract

Create `src/context/StudyPlanContext.js`. The `value` object is the contract: every screen will use these three names, and only the provider’s body changes later.

```
import { createContext, useContext } from 'react';
const StudyPlanContext = createContext(null);

export function StudyPlanProvider({ children }) {
  // TODO: replace these hard-coded values with real state
  const value = { selectedSteps: [], toggleStep: () => {}, removeStep: () => {} };
  return <StudyPlanContext value={value}>{children}</StudyPlanContext>;
}

export function useStudyPlan() {
  const context = useContext(StudyPlanContext);
  if (!context) throw new Error('useStudyPlan must be used inside StudyPlanProvider');
  return context;
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

A Hook That Fails Loudly

Screens call `useStudyPlan()`, not `useContext(StudyPlanContext)`. The context object itself isn't exported, so there's only one way to read it.

Without a provider above, `useContext` returns the default, `null`. A screen that destructures `null` crashes with a vague `Cannot destructure` error that doesn't say what's missing. The guard replaces that with an error that names the fix.

Try it before the provider is in place. Add this line to the top of `HomeScreen` in `index.js`, and save:

```
useStudyPlan(); // temporary: test the guard
```

The error screen shows `useStudyPlan must be used inside StudyPlanProvider`. Leave the line there for the next step.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Provide the Plan Above Every Screen

In `src/app/_layout.tsx`:

```
import { Stack } from 'expo-router';
import { StudyPlanProvider } from '../context/StudyPlanContext';

export default function Layout() {
  return (
    <StudyPlanProvider>
      <Stack>
        {/* the three Stack.Screen lines, unchanged */}
      </Stack>
    </StudyPlanProvider>
  );
}
```

Save. The error is gone, because Home is now inside the provider. Remove the temporary line from Home. The root layout is the closest component above all three screens.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Read the Plan From Context

In `details.js`, replace the `selectedSteps` state and the `toggleStep` function with one line, and send no params:

```
const { selectedSteps, toggleStep } = useStudyPlan();

function openPlan() {
  router.push('/plan');
}
```

In `plan.js`, replace `useLocalSearchParams` and the `planSteps` state:

```
const { selectedSteps: planSteps, removeStep } = useStudyPlan();
```

Renaming to `planSteps` leaves the rest of `plan.js` unchanged. Remove unused imports. Both screens now show 0, and Add does nothing: the context still holds the hard-coded value.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Keep the Plan’s State in the Provider

Replace the hard-coded `value` with the code that used to be in Study Steps, and add `useState` to the `react` import. Neither screen changes.

```
const [selectedSteps, setSelectedSteps] = useState([]);
function toggleStep(step) {
  setSelectedSteps((currentSteps) => {
    const alreadySelected = currentSteps.some((entry) => entry.id === step.id);
    if (alreadySelected) {
      return currentSteps.filter((entry) => entry.id !== step.id);
    }
    return [...currentSteps, step];
  });
}
function removeStep(id) {
  setSelectedSteps((currentSteps) => currentSteps.filter((step) => step.id !== id));
}
const value = { selectedSteps, toggleStep, removeStep };
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Test the Shared Plan

Repeat the steps that showed the problem:

Do this	Study Steps shows	Plan shows
Add three steps	3 in your plan	
Press Open plan		3 in your plan
Remove one step		2 in your plan
Go back	2 in your plan	
Press Open plan again		2 in your plan

There’s one array now, owned by the provider. Both screens read it, so a change on either screen appears on the other. When the provider’s state changes, React re-renders every component that calls `useStudyPlan`, including Study Steps while it’s covered by the plan.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Home Reads the Same Plan

In `index.js`, import the hook and read the plan:

```
import { useStudyPlan } from '../context/StudyPlanContext';

const { selectedSteps } = useStudyPlan();
```

Below the title:

```
<Text style={styles.body}>{selectedSteps.length} in your plan</Text>
```

Add steps, go back to Home, and check the count.

A third screen that needs the plan takes one import and one hook call. With params, Home would need a copy sent to it by every screen that can go back to it.

WE ARE ESSENTIAL TO ALBERTA | NAIT

What Stays Out of the Context

Moving the plan into a context doesn’t mean moving everything:

Value	Where it stays	Why
`status`, `data`, `errorMessage`	`useFetch`, in Study Steps	only Study Steps shows the request
text the user is typing in a form	the screen with the form	nothing else needs it until it’s submitted
whether a row is selected	computed in `renderStep`	it’s derived from the plan

Every component that reads a context re-renders when that context’s value changes. Put only the data that more than one screen needs into a context.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Browser’s Back Button Resets the Plan

In the browser, the browser’s own Back button remounts the app’s root layout. That unmounts the provider, and the plan resets to empty.

The back arrow in the app’s header and the Go back button keep the plan, because they pop the stack inside the app. On a phone, Android’s Back button and the iOS swipe gesture pop the stack too.

On the web, use the app’s own buttons today. The first challenge, saving the plan to storage, also fixes this.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Server State and Client State

Production apps split their state into two kinds, and use a different tool for each.

	Server state	Client state
Example	the study steps from the API	the plan the user is building
Owned by	the server. The app holds a copy that can go stale	the app
Needs	caching, refetching, retries, cancelling	sharing between screens, sometimes saving to the device
Common tools	TanStack Query, RTK Query, SWR	Context, Zustand, Redux Toolkit, Jotai

The TanStack Query docs call the client state left over after moving async data into a server-state library "usually very tiny". A context or two often covers it.

WE ARE ESSENTIAL TO ALBERTA | NAIT

When a Context Gets Expensive

React re-renders every component that reads a context when the provider’s value changes. It compares the old and new values with `Object.is`, so a new object counts as a change even when its contents are the same.

Our provider builds a new `value` object every time it renders. That’s fine here, because it only renders when the plan changes. A provider that re-renders for other reasons should wrap the value in `useMemo` and its functions in `useCallback`, as the React docs for `useContext` show.

A context that holds unrelated data makes every reader re-render for every change. Split it instead: a screen that reads only the signed-in user shouldn’t re-render when the plan changes.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Same Plan in Zustand

Zustand is a small client-state library used in many React Native apps. A store is a hook, created outside any component. There's no provider:

```
import { create } from 'zustand';
export const useStudyPlan = create((set) => ({
  selectedSteps: [],
  toggleStep: (step) => set((state) => ({
    selectedSteps: state.selectedSteps.some((entry) => entry.id === step.id)
      ? state.selectedSteps.filter((entry) => entry.id !== step.id)
      : [...state.selectedSteps, step],
  })),
  removeStep: (id) => set((state) => ({
    selectedSteps: state.selectedSteps.filter((step) => step.id !== id),
  })),
}));
```

Today's screens work with this store unchanged. The layout drops `<StudyPlanProvider>`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Reading a Zustand Store With a Selector

A component can pass the hook a **selector**, a function that picks the part of the store it needs:

```
const count = useStudyPlan((state) => state.selectedSteps.length);
```

The component re-renders only when the value its selector returns changes. If the store also held the user's theme, changing the theme wouldn't re-render a component that selects only the plan's count. A context can't make that distinction: every reader re-renders on every change to its value.

Zustand's `persist` middleware saves a store to AsyncStorage in a few lines. The first challenge does the same thing by hand, so you'll see what that middleware does for you.

Choose Context when a few screens share a small value that changes rarely. Reach for a store like Zustand when many components read different parts of a large, frequently changing state.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenges

Pick one of these challenges to finish after class. Each slide explains the problem briefly. Look up the term it names, read until you understand it, then implement a solution.

Each challenge gives you:

- what to do in the app to see the problem
- what your solution has to do
- a hint, and a starter snippet

Build it on top of your finished context, and commit it on its own, following the build order. The README has a full solution for each one in a collapsed section. Try it yourself before you look.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Save the Plan Between Launches

The plan lives in memory, so it’s gone whenever the app restarts. Look up **AsyncStorage**.

Trigger it: add three steps, then reload the app (press `r` in the Expo terminal, or refresh the browser). Home shows 0 in your plan.

Your solution: load the saved plan when the provider mounts, and save the plan whenever it changes. After a reload, Home shows the same count as before.

Install it with `npx expo install @react-native-async-storage/async-storage`. It’s a native module, so rebuild your development build afterwards (`npm run android:dev`). In the browser it uses `localStorage`, and needs no rebuild.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Save the Plan: Hint

Hint: use two effects in the provider: one that loads once, and one that saves whenever `selectedSteps` changes. The save mustn't run until the load has finished, or it writes the empty starting plan over the saved one.

```
const [isLoaded, setIsLoaded] = useState(false);

useEffect(() => {
  // TODO: read STORAGE_KEY. If something was saved, parse it into selectedSteps
  // TODO: then set isLoaded
}, []);

useEffect(() => {
  // TODO: if the saved plan has loaded, write selectedSteps to STORAGE_KEY as JSON
}, [selectedSteps, isLoaded]);
```

AsyncStorage stores only strings, so the plan goes in and out as JSON.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Show the Plan Count in the Header

Every screen should show the plan’s size, without each screen adding its own count. The header is drawn by the `Stack` in the layout. Look up **headerRight** in the Expo Router Stack docs.

Trigger it: open any screen. Nothing in the header shows the plan.

Your solution: show `Plan: 3` at the right end of every screen’s header, and keep it current as you add and remove steps.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Plan Count in the Header: Hint

Hint: `screenOptions` on `<Stack>` applies to every screen. `headerRight` takes a function that returns a component.

Calling `useStudyPlan` in `Layout` itself throws, because `Layout` renders the provider: it's above the provider, not inside it. Write a small component that calls the hook, and render that in the header, which is inside the provider.

```
function PlanBadge() {
  // TODO: read the plan from context
  // TODO: return <Text>Plan: {count}</Text>, with some right margin
}

<Stack screenOptions={{ headerRight: () => <PlanBadge /> }}>
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Clear the Plan With a Reducer

Each new action on the plan adds another function to the provider, each with its own `setSelectedSteps` call. Look up **useReducer**.

Trigger it: try to empty a plan of five steps. You have to remove them one at a time.

Your solution: move every change to the plan into one reducer function with `toggle`, `remove` and `clear` actions. Add a Clear plan button to the Plan screen, shown only when the plan isn't empty. The screens keep calling `toggleStep` and `removeStep` exactly as before.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Clear the Plan: Hint

Hint: a reducer takes the current state and an action, and returns the next state. The provider turns each function in the contract into a `dispatch` call, so the contract only grows by `clearPlan`.

```
function planReducer(selectedSteps, action) {
  switch (action.type) {
    case 'toggle': // TODO: toggleStep's logic, using action.step
    case 'remove': // TODO: filter out action.id
    case 'clear':  // TODO
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

const [selectedSteps, dispatch] = useReducer(planReducer, []);
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: A Step Screen From an Id

This is the job route params are good at. Look up **dynamic routes** in the Expo Router docs.

Trigger it: tap a step’s name on the Plan screen. Nothing happens.

Your solution: tapping a step’s name opens `/step/3`, which shows that step’s label and a Remove from plan button. Removing goes back to the plan, which now has one fewer step. Opening `/step/99` shows "Step 99 isn’t in your plan."

WE ARE ESSENTIAL TO ALBERTA | NAIT

Step Screen: Hint

Hint: a file named `src/app/step/[id].js` matches `/step/` followed by any id. The URL carries only the id. Look the step up in the plan from context, so the screen always shows current data.

```
export default function StepScreen() {
  const router = useRouter();
  const { id } = useLocalSearchParams();
  const { selectedSteps, removeStep } = useStudyPlan();

  // TODO: find the step whose id matches
  // TODO: show its label and a Remove from plan button,
  //       or "Step {id} isn't in your plan."
}
```

On the Plan screen, wrap each step's label in a `Pressable` that calls `router.push(`/step/${step.id}`)`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

- Name the three things that made the starter’s screens drift apart
- Why can’t `_layout.tsx` pass the plan to its screens as props?
- What does `useStudyPlan` do that calling `useContext` directly doesn’t?
- If the provider wrapped only the Study Steps screen, what would Home and Plan show, and why?
- Name one value that stays out of the context, and say why

WE ARE ESSENTIAL TO ALBERTA | NAIT

Before You Leave

1. Check that a change on Study Steps or Plan shows on both screens and on Home
2. Remove any temporary mount logs and test lines
3. Commit and push your work
4. Record your status: **resolved**, **partial**, or **blocked**
5. Pick a challenge, and do the exercise at the end of the lesson README

WE ARE ESSENTIAL TO ALBERTA | NAIT